

Hazard solving using "Forwarding"

Forwarding resolves data hazards by supplying the most recent value from **Ex/Mem** or **Mem/Wb** to the stage where the operand is consumed.

☐ Forwarding happens from **pipeline register** to a **stage input**.

☐ Valid forwarding wires — (As per our text book)

- (i) $Ex/Mem \rightarrow Ex$
 - (ii) $Mem/Wb \rightarrow Ex$
- } In case of multiple sources latest one gets priority.

Rules for Hazard solving using "Forwarding"

(i) Assuming you have figured out the hazards correctly.

(ii) Now you identify producer & consumer of the hazard.

Ex: (i) $Add\ X_5, X_7, X_8 \rightarrow$ This instruction is the producer of X_5 .

(ii) $Ld\ X_6, 16(X_5) \rightarrow$ This instruction is the consumer of X_5 .

(iii) Find the **stage of** consumer where you need the updated data as input.

Ex: (i) $Add\ X_5, X_7, X_8$; Producer

(ii) $Ld\ X_6, 16(X_5)$; consumer $\rightarrow$ here we need the data to calculate the address; So, the stage is Ex.

Ex: (i) $Add\ X_5, X_7, X_8$; Producer

(ii) $Sd\ X_5, 16(X_{12})$; consumer $\rightarrow$ here we need the data to store in memory; So, the stage is Mem.

(iv) Now we do the forwarding —

If a consumer needs an operand in the **Ex stage**, the value is taken from the **pipeline register** immediately to the left of the producer's current stage.

Ex1:

Add X₅, X₇, X₈; producer (i) Ex/Mem → Ex
Ld X₆, 16(X₅); consumer

	CC1	CC2	CC3	CC4	CC5	CC6
(i)	F	D	E	M	W	
(ii)		F	D	E	M	W

This is for understanding

	CC1	CC2	CC3	CC4	CC5	CC6
(i)	F	D	E	M	W	
(ii)		F	D	E	M	W

Do this in Exam

A special case:

Ex2: Ld X₅, 16(X₈) → producer
Add X₆, X₅, X₈ → consumer

	CC1	CC2	CC3	CC4	CC5	CC6
(i)	F	D	E	M	W	
(ii)		F	D	E	M	W

X

We get the value of X₅ at the end of CC4.

But we need that updated value of X₅ for the second instruction at the beginning of CC4.

So, a stall is a must here.

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
(i)	F	D	E	M	W		
(ii)		X	X	X	X	X	
(iii)			F	D	E	M	W

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
(i)	F	D	E	M	W		
(ii)		X	X	X	X	X	
(iii)			F	D	E	M	W

Hence, A stall is required only when a load is immediately followed by an instruction that needs the loaded value in the Ex Stage.

Ex3:

add X₅, X₆, X₇

add X₈, X₉, X₁₀

add X₇, X₅, X₈

(i) Mem/WB → Ex

(ii) Ex/Mem → Ex

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
(i)	F	D	E	M	W		
(ii)		F	D	E	M	W	
(iii)			F	D	E	M	W

Handwritten annotations in red: IF/CD, ID/EX, EX/MEM, MEM/WB. Arrows indicate data flow between stages. A red circle highlights the 'E' in CC5 of row (iii).

this is
for
understanding

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
(i)	F	D	E	M	W		
(ii)		F	D	E	M	W	
(iii)			F	D	E	M	W

Handwritten annotations in red: A red circle highlights the 'E' in CC5 of row (iii). Arrows indicate data flow between stages.

Do this in Exam:

Ex 4

add X_5, X_6, X_7 — no hazard
 add X_8, X_9, X_{10} — no hazard
 add X_7, X_5, X_8 — X_5, X_8
 ld $X_7, 12(X_7)$ — X_7
 add X_8, X_5, X_8 — no hazard
 sd $X_9, 12(X_7)$ — X_7

Using Forward to solve Data Hazard:

	cc1	cc2	cc3	cc4	cc5	cc6	cc7	cc8	cc9	cc10
(i)	F	D	E	M	W					
(ii)		F	D	E	M	W				
(iii)			F	D	E	M	W			
(iv)				F	D	E	M	W		
(v)					F	D	E	M	W	

Handwritten annotations in the table above indicate pipeline stages (IF, ID, EX, MEM, WB) and data hazards (E) with arrows showing the flow of data and the use of forward paths to resolve hazards.

(i) Mem/WB → Ex

(ii) Ex/Mem → Ex

(iii) Ex/Mem → Ex

(iv) Mem/WB → Ex

Ex/Mem → Ex } take the latest one
 Mem/WB → Ex }

	cc1	cc2	cc3	cc4	cc5	cc6	cc7	cc8	cc9	cc10
(i)	F	D	E	M	W					
(ii)		F	D	E	M	W				
(iii)			F	D	E	M	W			
(iv)				F	D	E	M	W		
(v)					F	D	E	M	W	
(vi)						F	D	E	M	W

Handwritten annotations in the table above show the resolved pipeline stages with arrows indicating the flow of data and the use of forward paths to resolve hazards.

Ex 5

Hazard(s)

add x_5 , x_1 , x_2 ; No

sub x_6 , x_5 , x_3 ; x_5

add x_7 , x_6 , x_5 ; x_5 , x_6

ld x_8 , 0(x_7) ; x_7

add x_9 , x_8 , x_6 ; x_8

add x_{10} , x_9 , x_5 ; x_9

sd x_{10} , 4(x_7) ; x_{10}

add x_{11} , x_{10} , x_8 ; No Hazard

sub x_{21} , x_{22} , x_{23} ; No Hazard

Assume we run the above instruction set in a pipelined datapath.

(i) Solve the data hazards using forwarding and stalling.

(ii) Determine the total clock cycle.

(iii) IF = 20ns ; ID = 20ns ; EX = 40ns ; Mem = 20ns ; Wb = 20ns

What will be the clock period?

(iv) Total time to run the instruction set.

Solve (i)

	cc1	cc2	cc3	cc4	cc5	cc6	cc7	cc8	cc9	cc10	cc11	cc12	cc13	cc14	cc15	cc16
(i)	IF	ID	EX	Mem	Wb											
(ii)		IF	ID	EX	Mem	Wb										
(iii)			IF	ID	EX	Mem	Wb									
(iv)				IF	ID	EX	Mem	Wb								
					X	X	X	X	X							
(v)						IF	ID	EX	Mem	Wb						
(vi)							IF	ID	EX	Mem	Wb					
								X	X	X	X	X				
									X	X	X	X	X			
(vii)										IF	ID	EX	Mem	Wb		
(viii)											IF	ID	EX	Mem	Wb	
(ix)												IF	ID	EX	Mem	Wb

(ii) Total Clock cycle = 16

(iii) Clock Period = 40s

(iv) Total Time = $(40 \times 16) = 640s$

Ex 6

add X_5, X_1, X_2 ; Hazard (s) No

sub X_6, X_5, X_3 ; X_5

add X_7, X_6, X_5 ; X_5, X_6

ld $X_8, 0(X_7)$; X_7

add X_9, X_8, X_6 ; X_8

add X_{10}, X_9, X_5 ; X_9

sd $X_{10}, 4(X_7)$; X_{10}

add X_{11}, X_{10}, X_8 ; No Hazard

sub X_{11}, X_{12}, X_{13} ; No Hazard

Assume we run the above instruction set in a pipelined datapath.

(i) Solve the data hazards using stall, forward & code scheduling.

(ii) Determine the total clock cycle.

(iii) IF = 20s ; ID = 20s ; Ex = 40s ; Mem = 20s ; Wb = 20s ;

What will be the clock period?

(iv) Total time to run the instruction set.

Solve (i)

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12	CC13	CC14	CC15
(i)	IF	ID	Ex	Mem	Wb										
(ii)		IF	ID	Ex	Mem	Wb									
(iii)			IF	ID	Ex	Mem	Wb								
(iv)				IF	ID	Ex	Mem	Wb							
					X	X	X	X	X						
(v)						IF	ID	Ex	Mem	Wb					
(vi)							IF	ID	Ex	Mem	Wb				
(vii)								IF	ID	Ex	Mem	Wb			
									X	X	X	X	X		
(viii)										IF	ID	Ex	Mem	Wb	
(viii)											IF	ID	Ex	Mem	Wb

(i) Total Clock cycle = 15

(iii) Clock Period = 40s

(iv) Total Time = $(40 \times 15) = 600s$

Bonus Content:

How to detect Hazard:

(i) Ignore the first line.

(ii) For the current instruction, list its source registers.

(iii) Compare these sources only with the destination registers of the previous two instructions.

(iv) If no match $\rightarrow$ No Hazard.

(v) If match $\rightarrow$ Data Hazard exists.

(vi) If both previous instructions update the same register, use the latest update.